

AFT (Another Frequency Thing)

Akshaya Tharun C R	Aneesh Kadiyala	Jakkala Naga Rohith Kumar
CS23BTECH11003	EE23BTECH11007	CS23BTECH11021

Rithvik Jami	Sai Rishi Kasturi
CS23BTECH11022	CS23BTECH11024

Muvvala Sairam Suhas	Panikar
CS23BTECH11038	CS23BTECH11040

Contents

1	Introduction to AFT	4
1.1	Background	4
1.2	Aim	4
2	General Syntax and Semantics	4
2.1	Primitive Data Types	4
2.1.1	Integers	4
2.1.2	Floating-Point Numbers	5
2.1.3	Complex Numbers	5
2.1.4	Boolean	5
2.1.5	Strings	5
2.1.6	Vectors	5
2.2	Keywords	6
2.3	Literals	6
2.3.1	Boolean Literals	6
2.3.2	Integer Literals	6
2.3.3	Float Literals	6
2.3.4	Complex Literals	7
2.3.5	Vector Literals	7
2.4	String Literals	7
2.5	Identifiers	7
2.6	Operators	8
2.6.1	Unary Operators	8
2.6.2	Arithmetic Operators	8
2.6.3	Comparison Operators	8
2.6.4	Logical Operators	8
2.6.5	Bitwise Operators	8
2.6.6	String Operators	9
2.6.7	Vector Operators	9
2.6.8	Type Casting with <code>as</code>	9
2.6.9	Operator Precedence	10
2.7	Variable Declaration	10
2.8	Control Flows	11
2.8.1	Conditional Statements	11
2.8.2	Loops	11
2.9	Functions	12
2.10	Structs	12
2.11	Comments	13
3	Built-in Functions	13
3.1	Basic Mathematical Functions	13
3.2	Trigonometric Functions	13
3.3	Complex Number Functions	13
3.4	Vector Functions	13
3.5	Signal Processing Functions	14
3.6	Debugging Functions	14
3.7	File I/O	14

4	Example Code	15
4.1	Basic Signal Generation	15
4.2	Complex Number Operations	15
4.3	Arbitrary LTI Filter	15
4.4	Lowpass Butterworth Filter	16
4.5	Spectral Analysis	16

1 Introduction to AFT

1.1 Background

AFT is a domain-specific language designed specifically for signal processing. It supports a rich set of foundational types such as integers, floats, complex numbers, vectors, matrices, and discrete signals. Planned operations include scaling, shifting, arithmetic, logical and bitwise operations, as well as domain-specific constructs like filters and transforms.

Existing tools like MATLAB and Octave are widely used in the signal-processing community, but they function primarily as simulation environments. Their interpreted nature limits performance, and they do not enforce strong type safety. AFT addresses these limitations by offering a statically typed system with opportunities for domain-specific optimization. It also aims to express signal-processing concepts more naturally than general-purpose programming languages.

1.2 Aim

The primary aim of AFT is to bridge the gap between textbook-style mathematical notation and executable implementation. By providing high-level abstractions aligned closely with established signal-processing theory, AFT enables developers and researchers to write code that is both clear and efficient. The language intends to unify the expressiveness of mathematical notation with the performance and safety of a compiled, domain-specialized language.

2 General Syntax and Semantics

2.1 Primitive Data Types

2.1.1 Integers

AFT provides a complete set of integer types with explicit bit-width specifications.

Unsigned Integers:

Type	Size	Minimum Value	Maximum Value
u8	8-bit	0	255
u16	16-bit	0	65535
u32	32-bit	0	$2^{32} - 1$
u64	64-bit	0	$2^{64} - 1$
u128	128-bit	0	$2^{128} - 1$

Signed Integers:

Type	Size	Minimum Value	Maximum Value
i8	8-bit	-128	127
i16	16-bit	-32768	32767
i32	32-bit	-2^{31}	$2^{31} - 1$
i64	64-bit	-2^{63}	$2^{63} - 1$
i128	128-bit	-2^{127}	$2^{127} - 1$

By default, any integer literals without an explicitly mentioned type will be inferred as **i64**.

Table 1: Caption

2.1.2 Floating-Point Numbers

AFT provides IEEE 754-compliant floating-point types for real number computations, with support for both single and double precision arithmetic.

Type	Size	Mantissa	Min. Exp.	Max. Exp.
<code>f32</code>	32-bit	24-bit	-126	127
<code>f64</code>	64-bit	53-bit	-1022	1023

By default, any floating-point literals without an explicitly mentioned type will be inferred as `f64`.

2.1.3 Complex Numbers

AFT provides native support for complex numbers, essential for signal processing. Complex numbers are represented as pairs of floating-point values for the real and imaginary components.

Type	Real Part	Imaginary Part
<code>c32</code>	<code>f32</code>	<code>f32</code>
<code>c64</code>	<code>f64</code>	<code>f64</code>

By default, complex number literals without explicit typing are inferred as `c64`.

2.1.4 Boolean

AFT provides a standard boolean type for logical operations and control flow.

Type: `bool`

2.1.5 Strings

AFT provides a standard type for strings. Strings accept all valid ASCII characters. No provisions have been made to work with Unicode characters or characters from other character sets.

2.1.6 Vectors

AFT provides built-in support for homogeneous collections through vector types, enabling efficient storage and manipulation of sequences of elements.

Type: `[T]`

Vectors can also be initialized using ranges:

```
let x = a..b; // [ a, a + 1, a + 2, ..., b - 1 ]
let x = a..=b; // [ a, a + 1, a + 2, ..., b ]
```

2.2 Keywords

AFT reserves the following keywords for language constructs:

<code>let</code>	<code>const</code>	<code>fn</code>	<code>return</code>
<code>if</code>	<code>else</code>	<code>while</code>	<code>for</code>
<code>repeat</code>	<code>struct</code>	<code>enum</code>	<code>true</code>
<code>false</code>	<code>as</code>	<code>i8</code>	<code>u8</code>
<code>i16</code>	<code>u16</code>	<code>i32</code>	<code>u32</code>
<code>i64</code>	<code>u64</code>	<code>i128</code>	<code>u128</code>
<code>f32</code>	<code>f64</code>	<code>c32</code>	<code>c64</code>
<code>bool</code>	<code>str</code>	<code>continue</code>	<code>break</code>
<code>return</code>	<code>static</code>	<code>PI</code>	

2.3 Literals

2.3.1 Boolean Literals

- `true`
- `false`

2.3.2 Integer Literals

- **Decimal (default):** 42, 123
- **Binary:** 0b1010, 0b11110000
- **Octal:** 0o755, 0o644
- **Hexadecimal:** 0xFF, 0xDEADBEEF

2.3.3 Float Literals

The integer part can be omitted if it is 0. The decimal part can only be omitted if it is zero and an exponent is present.

- `PI` - 3.14149... , pi is available as float const
- `.5` - Integer part omitted
- `2.0`
- `0.5`
- `2.5e10` - Scientific notation
- `1e6` - Scientific notation with omitted decimal part
- `.3e-6` - Scientific notation with omitted integer part
- Various ways to write zero: `.0`, `0.0`

2.3.4 Complex Literals

The imaginary part needs to be explicitly specified for the literal to be inferred as complex.

- `3.0 + 4.0j` - Real and imaginary parts
- `2.5j` - Pure imaginary number
- `0+0j` - Complex zero
- `6 - 9j` - Negative imaginary part
- `6 - j` - imaginary part without constant

2.3.5 Vector Literals

Vector literals include comma-separated elements enclosed in square brackets.

- `[1, 2, 3]` - Integer vector
- `[1.0, 2.0, 3.0]` - Float vector
- `[1, 2j, -3, -4j]` - Complex vector

2.4 String Literals

String literals must be enclosed in double quotes `"`. AFT supports the following escape sequences in string literals:

Escape Sequence	Character Represented
<code>\n</code>	Line Feed (LF)
<code>\r</code>	Carriage Return (CR)
<code>\t</code>	Horizontal Tab
<code>\v</code>	Vertical Tab
<code>\\</code>	Backslash
<code>\"</code>	Double Quotation Mark

```
// This is a sample "string".
let s = "This is a sample \"string\".";
```

2.5 Identifiers

Identifiers in AFT are case-sensitive and must follow these rules:

- Must start with an alphabetic character or underscore
- Can contain alphanumeric characters and underscores
- Cannot be a reserved keyword

Valid identifiers: `variable_name`, `_private`, `camelCase`, `CONSTANT_VALUE`

2.6 Operators

2.6.1 Unary Operators

Operator	Description
+	positive
-	negative

2.6.2 Arithmetic Operators

Operator	Description
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulo
**	Exponentiation

2.6.3 Comparison Operators

Operator	Description
==	Equal to
!=	Not equal to
<	Less than
>	Greater than
<=	Less than or equal to
>=	Greater than or equal to

2.6.4 Logical Operators

All the logical operators are short-circuit operators. They can only be applied to values of type `bool`.

Operator	Description
&&	Logical AND
	Logical OR
^	Logical XOR
!	Logical NOT

2.6.5 Bitwise Operators

AFT offers standard bitwise operators that can be applied to any integer type. Both operands of binary bitwise operators must be of the same type.

Operator	Description
&	Bitwise AND
	Bitwise OR
^	Bitwise XOR
<<	Bitwise left shift
>>	Bitwise right shift
!	Bitwise NOT

2.6.6 String Operators

Operator	Description
+	Concatenation
[]	Subscripting

Note: Strings are 0-indexed.

2.6.7 Vector Operators

Operator	Description
	Concatenation
.*	Convolution
+	Vector Addition (or Unary)
-	Vector Subtraction (or Unary)
*	Scalar Multiplication
/	Scalar Division
[]	Subscripting
~	Reversing the order of elements

Note: Vectors are 0-indexed.

2.6.8 Type Casting with as

In AFT, explicit type casting is done using the `as` keyword .

Expression	Description
<code>x as i32</code>	Casts <code>x</code> to integer
<code>y as f32</code>	Casts <code>y</code> to floating-point
<code>z as c64</code>	Casts <code>z</code> to complex number
<code>v as [c64]</code>	Converts vector <code>v</code> into a vector of <code>c64</code> type

Example usage:

```
a = 3.7
b = a as i32      # b becomes 3
c = b as f32      # c becomes 3.0
```

2.6.9 Operator Precedence

Operators	Associativity
Function Calls Vector and String Subscripting Struct Member Access (.)	left-to-right
Unary Operators (+, -, ~) Bitwise NOT Logical NOT	right-to-left
Type Casting	left-to-right
**	right-to-left
, /, %, .	left-to-right
+, -	left-to-right
<<, >>	left-to-right
&	left-to-right
^	left-to-right
	left-to-right
Comparison Operators	-
&&	left-to-right
Logical OR	left-to-right
Vector Concatenation	left-to-right
=	right-to-left
,	left-to-right
return	-

2.7 Variable Declaration

AFT supports type inference and explicit type annotations for variable declarations:

```

let a = 123;                // i32 (inferred)
let b: u8 = 255;            // explicit u8 type
let c: [c32] = [1, 2j, -3, -4j]; // complex array
let d: [c32] = [1, 2j, -3, -4j]; // same as above
let e = [1, 2j, -3, -4j];    // type inferred as [c64]
let f = 6 - 9j;              // type inferred as c64

let complex_64_array = []; // ERROR: Type cannot be inferred

// Correct approach:
let complex_64_array: [c64] = [];

let x = 3.0;
x = 2; // ERROR: Assigning integer to float

let x = 3.0;
let x = 2; // WARNING: Variable x is shadowed

let a = 0+0j; // Complex zero (c64)
let a = 0.;   // Float zero (f64)
let a = 0;    // Integer zero (i32)

let a = 1, b = 2.0, c = 4 + j ;

```

Multiple variables can be declared in a single statement:

```
let (a,b,c) = (1,2.0,4+j) ;
let (a,b,a) = (1,2.0,4+j) ;           // ERROR : Attempt to shadow variable in same statement
```

Parentheses are allowed on the left-hand side of variable declarations and assignment operators only if there are two or more comma-separated identifiers present. Constants can be defined using the `const` keyword. The type must be explicitly mentioned for constants.

```
// Definition of PI (a built-in constant)
const PI: f64 = 3.14159265358979323846264338327950288;
const TAU: f64 = 2. * PI;
const (PI_2, PI_4): (f64, f64) = (PI / 2., PI / 4.);
```

The right-hand side of a constant definition must only have literals and other constant variables. Only `const` variables can be global variables.

2.8 Control Flows

2.8.1 Conditional Statements

```
if condition {
    // do something
}

if condition {
    // do something
} else {
    // do something else
}

if condition1 {
    // do something
} else if condition2 {
    // otherwise do something
} else {
    // do something else
}
```

2.8.2 Loops

Repeat Loop

```
repeat number_of_iterations {
    // do something
}
```

`number_of_iterations` must be an unsigned integer.

While Loop

```
while condition {
    // do something
}
```

For Loop

```
for element in collection {
    // do something
}
```

2.9 Functions

Functions in AFT are defined as below:

```
fn function_name(parameter_list) return_type_list {
    // do something
}
```

Calling functions is straightforward:

```
function_name(param1, param2); // Example function call for function with 2 arguments
let x = function_with_1_return(...);
let (x, y) = function_with_2_return(...);
```

Functions can return multiple values. Returned values must be enclosed in parentheses and separated by commas.

```
fn sum_and_product(a: i32, b: i32) (i32, i32) {
    return (a + b, a * b);
}
let (x, y) = sum_and_product(3, 2);
```

When assigning a function's returned values to variables, the number of variables must be equal to the number of returned values.

```
(x, y) = sum_and_product(3, 2); // OK
x = sum_and_product(3, 2); // ERROR:
```

2.10 Structs

AFT supports user-defined structs with members laid out in memory in the same order they are declared. These structs also support nesting.

```
// A simple struct with three 32-bit integers as members
struct Point3 {
    x: i32,
    y: i32,
    z: i32
}
```

Structs can be initialized as follows:

```
// Create a Point3 instance with x = 1, y = 2 and z = 3.
let point_1 = Point3 { x: 1, y: 2, z: 3 };
```

Struct members can be accessed using the `.` operator.

```
let x_coordinate_of_point_1 = point_1.x;
```

2.11 Comments

AFT supports both single-line and multi-line comments:

```
// This is a single line comment
```

```
/*
This is a
multi-line
comment.
*/
```

Single-line comments begin with `//` and continue to the end of the line. Multi-line comments are enclosed between `/*` and `*/` and can span multiple lines.

3 Built-in Functions

3.1 Basic Mathematical Functions

Function	Description
<code>abs(x)</code>	Absolute Value
<code>sqrt(x)</code>	Square Root
<code>pow(x, y)</code>	x^y
<code>exp(x)</code>	Exponential Function
<code>ln(x)</code>	Natural Logarithm
<code>log10(x)</code>	Base-10 Logarithm
<code>log2(x)</code>	Base-2 Logarithm

3.2 Trigonometric Functions

AFT offers the following trigonometric functions: `sin`, `cos`, `tan`, `sec`, `cosec`, `cot`, `arcsin`, `arccos`, `arctan`.

3.3 Complex Number Functions

Function	Description
<code>conj(z)</code>	Complex Conjugate
<code>magnitude(z)</code>	Magnitude
<code>arg(z)</code>	Argument

3.4 Vector Functions

Function	Description
<code>len(vec)</code>	Vector Length
<code>sum(vec)</code>	Sum of Elements
<code>mean(vec)</code>	Arithmetic Mean
<code>min(vec)</code>	Minimum Element
<code>max(vec)</code>	Maximum Element
<code>norm(vec)</code>	L-2 Norm of the Vector

Here, `vec` must be a vector. `sum`, `mean` and `norm` can only be called on integer, floating-point and complex vectors. `min` and `max` can only be called on integer and floating-point vectors.

3.5 Signal Processing Functions

Function	Description
<code>fft(x)</code>	Fast Fourier Transform
<code>ifft(x)</code>	Inverse Fast Fourier Transform
<code>correlate(x, y)</code>	Cross-correlation
<code>resample(x, fs1, fs2)</code>	Resampling
<code>filter(b, a, x)</code>	Filter
<code>lowpass(...)</code>	Low-pass Filter
<code>highpass(...)</code>	High-pass Filter
<code>bandpass(...)</code>	Band-pass Filter
<code>bandstop(...)</code>	Band-stop Filter

3.6 Debugging Functions

AFT offers a function called `dbg` that takes variable number of inputs of any type.

Example Usage

```
dbg("x is ", x, "\ny is ", y);
/* Output
   x is ...
   y is ...
*/
```

3.7 File I/O

Function	Description
<code>load(filename)</code>	Reads values from a file into a vector
<code>save(filename, x, append)</code>	Writes values from vector x into a file. The file contents will be overwritten unless append is <code>true</code> .

4 Example Code

4.1 Basic Signal Generation

```
fn generate_sine_wave(frequency: f64, sample_rate: f64, duration: f64) [f64] {
    let num_samples = (duration * sample_rate) as u32;
    let signal: [f64] = [];

    for i in 0..num_samples {
        let t = i as f64 / sample_rate;
        let sample = sin(2.0 * PI * frequency * t);
        signal = signal || [sample]; // Concatenate
    }

    return signal;
}

fn main() {
    let fs = 44100.0; // Sample rate
    let freq = 440.0; // A4 note
    let duration = 1.0; // 1 second

    let sine_wave = generate_sine_wave(freq, fs, duration);
    dbg(sine_wave);
}
```

4.2 Complex Number Operations

```
fn complex_arithmetic_demo() {
    // Basic complex number operations
    let z1 = 3.0 + 4.0j;
    let z2 = 1.0 - 2.0j;
    let sum = z1 + z2; // 4.0 + 2.0j
    let product = z1 * z2; // 11.0 - 2.0j
    let conjugate = conj(z1); // 3.0 - 4.0j

    dbg(z1, z2, sum, product, conjugate);

    // Polar form conversion
    let magnitude = abs(z1); // 5.0
    let angle = arg(z1); // 0.927 radians
    dbg(magnitude, angle);
}
```

4.3 Arbitrary LTI Filter

```
fn arbitrary_LTI(h: [f64], x: [f64]) [f64] {
    return x .* h;
}
```

4.4 Lowpass Butterworth Filter

```
fn lowpass_butterworth(cutoff: f64, sample_rate: f64, signal: [f64]) [f64] {
    let omega = 2.0 * PI * cutoff / sample_rate;
    let cos_omega = cos(omega);
    let sin_omega = sin(omega);
    let alpha = sin_omega / (2.0 * 0.707);
    // Filter coefficients
    let b0 = (1.0 - cos_omega) / 2.0;
    let b1 = 1.0 - cos_omega;
    let b2 = (1.0 - cos_omega) / 2.0;
    let a0 = 1.0 + alpha;
    let a1 = -2.0 * cos_omega;
    let a2 = 1.0 - alpha;
    // Normalize the coefficients.
    let b = [b0, b1, b2] / a0;
    let a = [a0, a1, a2] / a0;
    // Apply the filter.
    return filter(b, a, signal);
}
```

4.5 Spectral Analysis

```
fn spectral_analysis_example() {
    let fs = 1000.0; // 1 kHz sampling rate
    let duration = 2.0; // 2 seconds
    let num_samples = (fs * duration) as u32;

    // Generate test signal: sum of sinusoids
    let signal: [f64] = [];
    let t = (0..num_samples) as [f64] / fs;
    let s = sin(2.0 * PI * 50.0 * t) // 50 Hz component
        + 0.5 * sin(2.0 * PI * 120.0 * t) // 120 Hz component
        + 0.1 * sin(2.0 * PI * 300.0 * t); // 300 Hz component

    // Compute FFT
    let spectrum = fft(s);

    // Calculate magnitude spectrum
    let magnitude_spectrum = abs(spectrum);

    // Find peaks in spectrum
    let peak_indices = find_peaks(magnitude_spectrum, 0.1);

    dbg("Frequency peaks at ");
    for idx in peak_indices {
        let frequency = idx as f64 * fs / len(spectrum) as f64;
        dbg(frequency);
    }
}

fn find_peaks(data: [f64], threshold: f64) {
```

```
let peaks: [u32] = [];  
  
for i in 1..(len(data) - 1) {  
    if data[i] > data[i-1] && data[i] > data[i+1] && data[i] > threshold {  
        peaks = peaks || [i as u32];  
    }  
}  
  
return peaks;  
}
```